

HelixMemory

HelixMemory

Единая память для агентов AI — четыре лучших движка, объединённые в один.

Краткое описание

HelixMemory — это Go SDK, объединяющий четыре ведущие системы памяти (Mem0, Cognition, Letta, Graphiti) в единый когнитивный движок памяти, осуществляющий параллельный поиск и слияние результатов. Он предоставляет приложениям AI один надёжный, дедуплицированный и переранжированный слой памяти вместо четырёх разрозненных.

Краткая характеристика

HelixMemory — это Go SDK, объединяющий Mem0, Cognition, Letta и Graphiti в единый когнитивный движок памяти для приложений AI. Он интеллектуально распределяет записи, осуществляет параллельный поиск по всем бэкендам и объединяет результаты с помощью трёхэтапного конвейера: сбор — дедупликация — переранжирование.

Подробное описание

HelixMemory — это унифицированный когнитивный движок памяти для приложений AI, реализованный в виде Go SDK (модуль `digital.vasic.helixmemory`, Go версии 1.25 и выше). Его основная идея заключается в том, что ни один отдельный проект памяти никогда не будет идеален во всём. Вместо того чтобы заново реализовывать память с нуля и наследовать ограничения одного проекта, HelixMemory оркестрирует четыре лучшие системы, позволяя каждой раскрыть свои сильные стороны: Mem0 для динамического извлечения фактов и управления предпочтениями, Cognition для семантических графов знаний, построенных с помощью ECL-

конвейеров, Letta для среды исполнения агентов с редактируемыми блоками памяти и вычислениями в режиме сна, а также Graphiti для битемпорального графа знаний, позволяющего анализировать изменения фактов во времени.

Движок слияния превращает эти четыре независимых хранилища в единый мозг. При записи каждое поступающее воспоминание классифицируется по содержанию и направляется в наиболее подходящий бэкенд. При чтении запрос параллельно рассылается по всем бэкендам, а полученные результаты проходят через трёхэтапный конвейер слияния: сбор, дедупликация и переранжирование с учётом источников. Благодаря этому вызывающая сторона получает не четыре зашумлённых и пересекающихся набора результатов, а один чистый и ранжированный ответ. Каждый бэкенд обёрнут в предохранитель: если один из движков выходит из строя, его предохранитель срабатывает, а оставшиеся бэкенды продолжают обслуживание, не утягивая за собой весь слой памяти. Поскольку движок реализует интерфейс MemoryStore, его можно использовать как прямую замену обычного провайдера памяти — без необходимости перепроектировать вызывающую сторону. Метрики Prometheus обеспечивают полную наблюдаемость процессов маршрутизации и слияния.

HelixMemory был создан как слой памяти для HelixAgent — более широкого ансамбля Helix AI — и перенял семейную дисциплину антиблефового тестирования в область памяти. Встроенный тестовый раннер проверяет реальные производственные пути выполнения кода: маршрутизацию, слияние, транслятор, предохранители, — а обёртка с мутациями намеренно нарушает инварианты, чтобы доказать, что тесты действительно падают при нарушении логики. Зелёный прогон тестов в этом случае имеет реальный смысл.

Зачем мы его создали

Агентам AI необходима долговечная и высококачественная память, однако экосистема фрагментирована — каждый проект памяти (Mem0, Cognee, Letta, Graphiti) силён в чём-то одном и уязвим в других аспектах. HelixMemory был создан, чтобы предоставить HelixAgent единую поверхность памяти, объединяющую их сильные стороны без привязки к какому-либо одному из них.

Почему это меняет правила игры

Это устраняет вынужденный выбор. Четыре системы памяти, обычно конкурирующие за одно и то же место, становятся взаимодополняющими бэкендами за единым интерфейсом — так приложение получает динамическое извлечение фактов, семантические графы знаний, память с сохранением состояния агентов и битемпоральные рассуждения *одновременно*, с автоматическим устранением дубликатов и переранжированием из разных источников. То, что раньше было непрактично, теперь не ставит перед ложной дилеммой: «Какую систему памяти выбрать?» HelixMemory позволяет использовать все их преимущества разом, через единый готовый к подключению MemoryStore, не наследуя при этом слепых зон какого-либо одного движка и не привязываясь к нему навсегда.

Что нового

- **Мультибэкендный синтез** (сбор → дедупликация → переранжирование из разных источников), возвращающий один ранжированный набор результатов, вместо того чтобы привязывать вызывающую сторону к одному хранилищу.
- **Интеллектуальная маршрутизация записей**, классифицирующая каждую память по содержимому и направляющая её в движок, наиболее подходящий для её хранения, — так нужные данные попадают в нужное хранилище.
- **Плавная деградация** благодаря предохранителям для каждого бэкенда — сбой одного движка изолируется, не становясь критическим, а остальные продолжают обслуживание.
- **Консолидация вычислений в период простоя** (через Letta), перерабатывающая память в фоновом режиме, а не только во время запросов.
- **Антиблефовая верификация**: запуск тестов на реальном производственном коде в сочетании с обёрткой мутаций, которая должна завершаться ошибкой при нарушении инварианта, — это доказывает, что проверка действительно работает, а не является тавтологией.

Крупнейшие технические вызовы и их решения

- **Объединение четырёх разнородных бэкендов в единый согласованный набор результатов** — каждый движок возвращает память в своём формате, и наивное слияние приводит к дубликатам и несопоставимым

ранжированиям. Решение: типизированный движок синтеза, который собирает данные из всех источников, устраняет пересечения, переранжирует всё по единой шкале, а в тестах проверяется инвариант объединённого количества, чтобы слияние не теряло и не дублировало результаты.

- **Обеспечение работоспособности при отказе бэкенда** — недоступность одного движка памяти не должна парализовать весь слой. Решение: предохранители для каждого бэкенда, реализующие конечный автомат закрытое → открытое (после превышения порога сбоев) → полуоткрытое (после тайм-аута) состояние, изолирующий проблемный бэкенд и продолжающий обслуживание на исправных, пока он не восстановится.
- **Доказательство работоспособности логики памяти, а не только её компилируемости** — зелёный набор тестов бесполезен, если тесты не могут упасть. Решение: встроенный тестовый раннер, запускающий реальный производственный код (маршрутизацию, синтез, транслятор, предохранители), и парная обёртка мутаций, которая меняет инварианты и требует, чтобы тесты завершались ошибкой, — так проверка гарантированно не является тавтологией.

Технологический стек

- **Go (1.25+)** — единый SDK и рантайм; выбран, поскольку параллельное чтение с распределением нагрузки между четырьмя бэкендами — это задача конкурентности, а горутины Go делают её экономичной, в то время как интерфейсные типы обеспечивают всей системе единую чёткую точку интеграции (MemoryStore), на которую могут полагаться вызывающие модули.
- **Mem0** — динамический бэкенд для извлечения фактов и управления предпочтениями; используется для работы с фрагментом памяти, отвечающим за «что на самом деле предпочитает пользователь / какие факты всплыли».
- **Cogne** — бэкенд семантического графа знаний, построенный на ECL-конвейерах; предназначен для хранения структурированных, взаимосвязанных знаний, а не плоских фактов.
- **Letta** — бэкенд с сохраняемым состоянием для исполнения агентов, поддерживающий редактируемые блоки памяти и вычисления в периоды простоя; применяется там, где память должна сохраняться как активное состояние агента и консолидироваться в периоды бездействия.
- **Graphiti** — битемпоральный бэкенд графа знаний; используется для рассуждений о том, как факты и связи меняются со временем, а не только об их текущем состоянии.

- **PostgreSQL + Neo4j + Redis** — реальные хранилища данных, на которых работают бэкенды; развёрнуты для полноценного интеграционного тестирования с помощью `make infra-start`, чтобы тестовый набор проверял реальную инфраструктуру, а не заглушки.
- **Prometheus** — метрики и наблюдаемость, интегрированные через конвейер слияния, благодаря чему маршрутизация и поведение системы слияния поддаются измерению в продакшене, а не остаются «чёрным ящиком».
- **Шов для переводов i18n** — именованная (`helixmemory_`) поверхность строк, сохранённая для того, чтобы любой будущий пользовательский слой можно было локализовать без переработки ядра.

Статус и замечания о прозрачности

- **Статус: бета-версия.** Работающий SDK; разработан как слой памяти для HelixAgent.
- **Лицензия: не определена.** Файл LICENSE не обнаружен через GitHub API — НЕПРОВЕРЕНО / не объявлено.
- Отображаемое название «HelixMemory» соответствует репозиторию `memory`. Приведённые в README показатели точности заимствованы у сторонних поставщиков, а не измерены в рамках HelixMemory, и здесь не приводятся.

Приоритетный уровень: первоочередной для Helix.